

LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:

Nama: Jovan Christo Alvaro (2509106031)

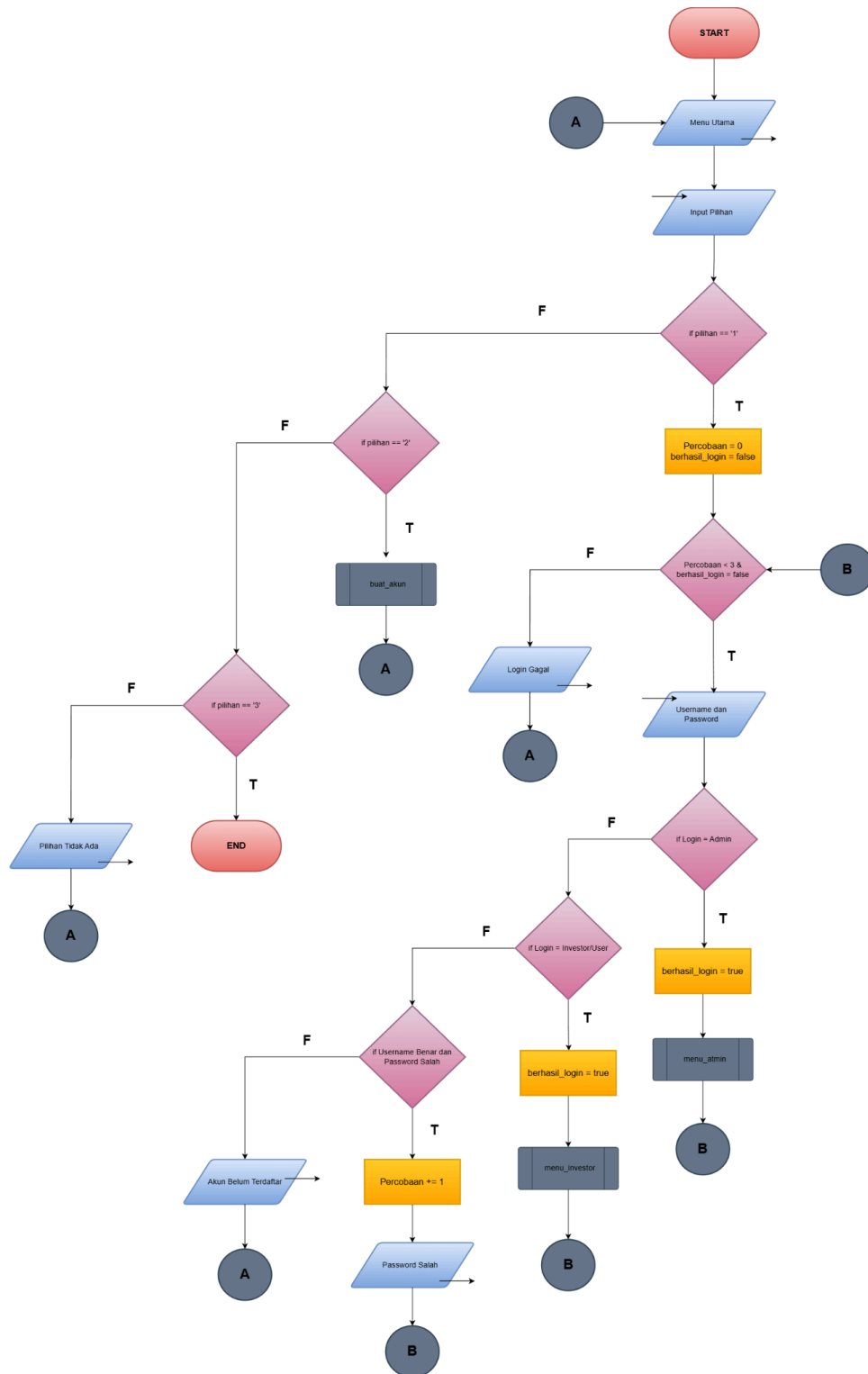
Kelas: (A2)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA

2025

1. Flowchart

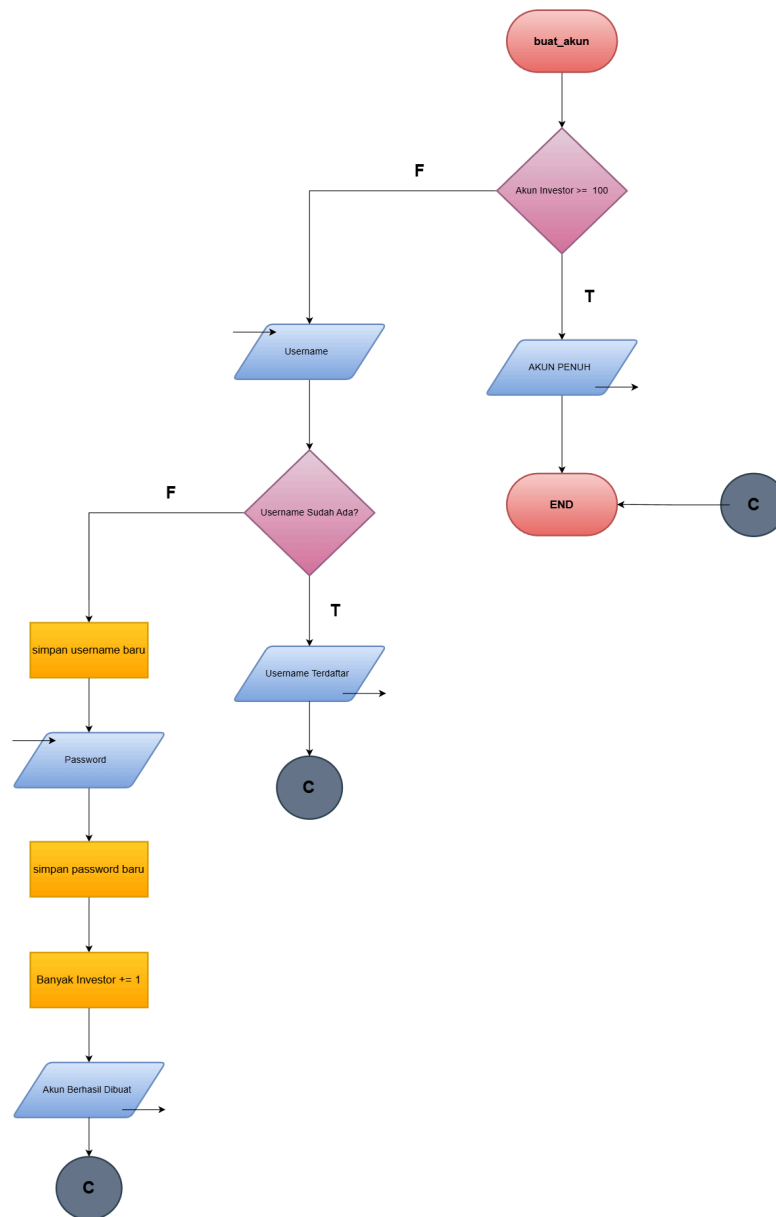
A. Flowchart Utama



Gambar 1.A. Flowchart Utama

Pada bagian Flowchart diatas adalah sistem **Menu Utama** yang berjalan terus-menerus (looping) sampai user memilih **Keluar (Menu 3)**. Saat user pilih **Login (Menu 1)**, program menjalankan **batas 3x percobaan** di dalamnya, sistem akan mengecek apakah user adalah **Admin** atau **Investor/User**. Jika berhasil login, program memanggil **Sub-Program** (kotak garis ganda) sesuai role-nya, lalu setelah selesai, status login diubah jadi **True** untuk memutus perulangan login dan kembali ke Menu Utama. Selain itu, ada **Menu 2** untuk mendaftarkan/membuat akun baru melalui fungsi **buat_akun** sebelum akhirnya bisa digunakan untuk login.

B. Flowchart Prosedur Buat Akun

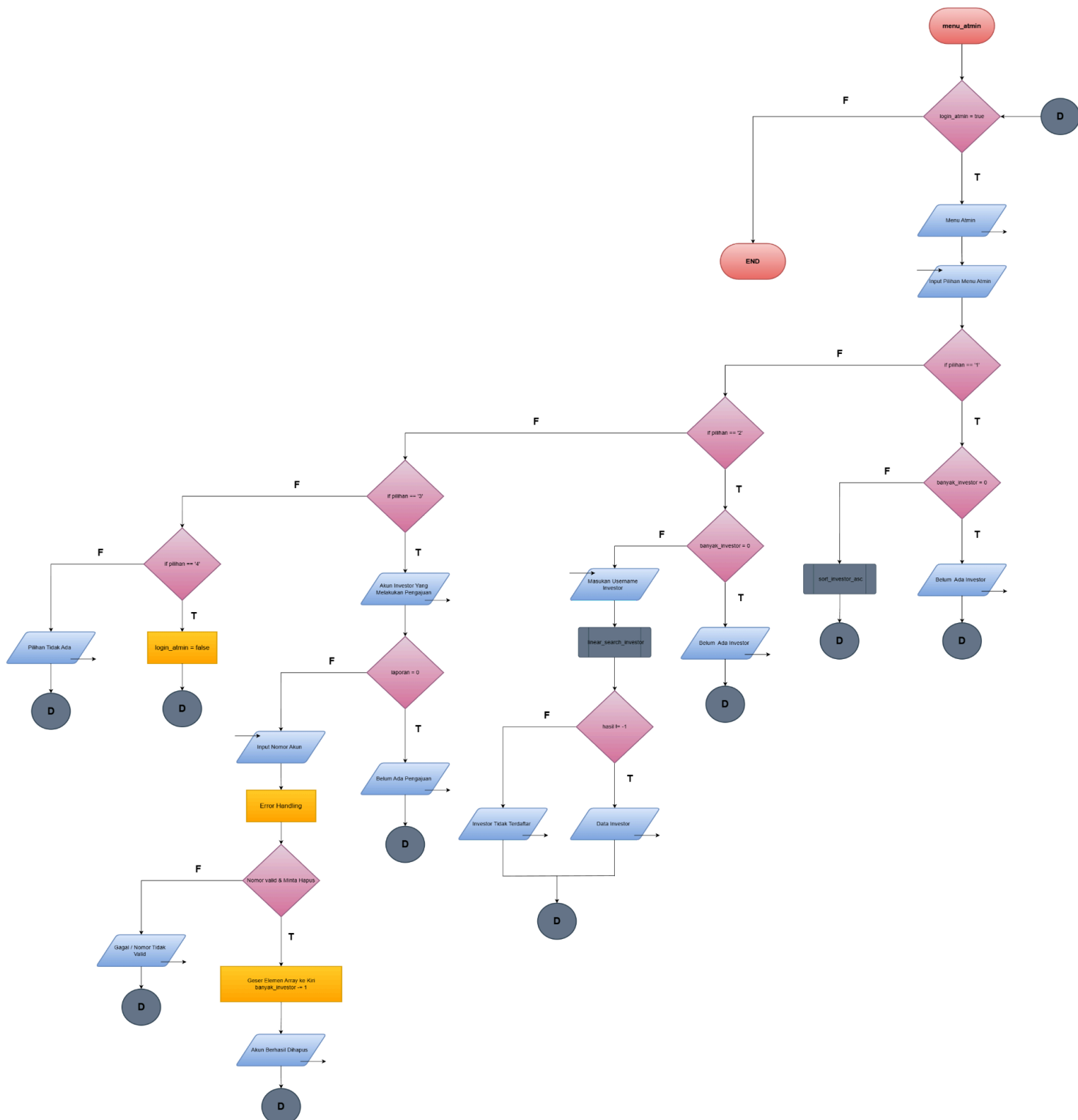


Gambar 1.B. Flowchart Buat Akun

Flowchart diatas bertugas memvalidasi pendaftaran user baru dengan alur → cek kapasitas maksimal (100 akun), verifikasi apakah username sudah terdaftar atau belum (untuk menghindari duplikasi), lalu jika semua syarat terpenuhi, program akan menyimpan data username dan password baru, menambah jumlah investor, dan

memberikan pesan sukses sebelum kembali ke menu utama melalui konektor C.

C. Flowchart Menu Admin

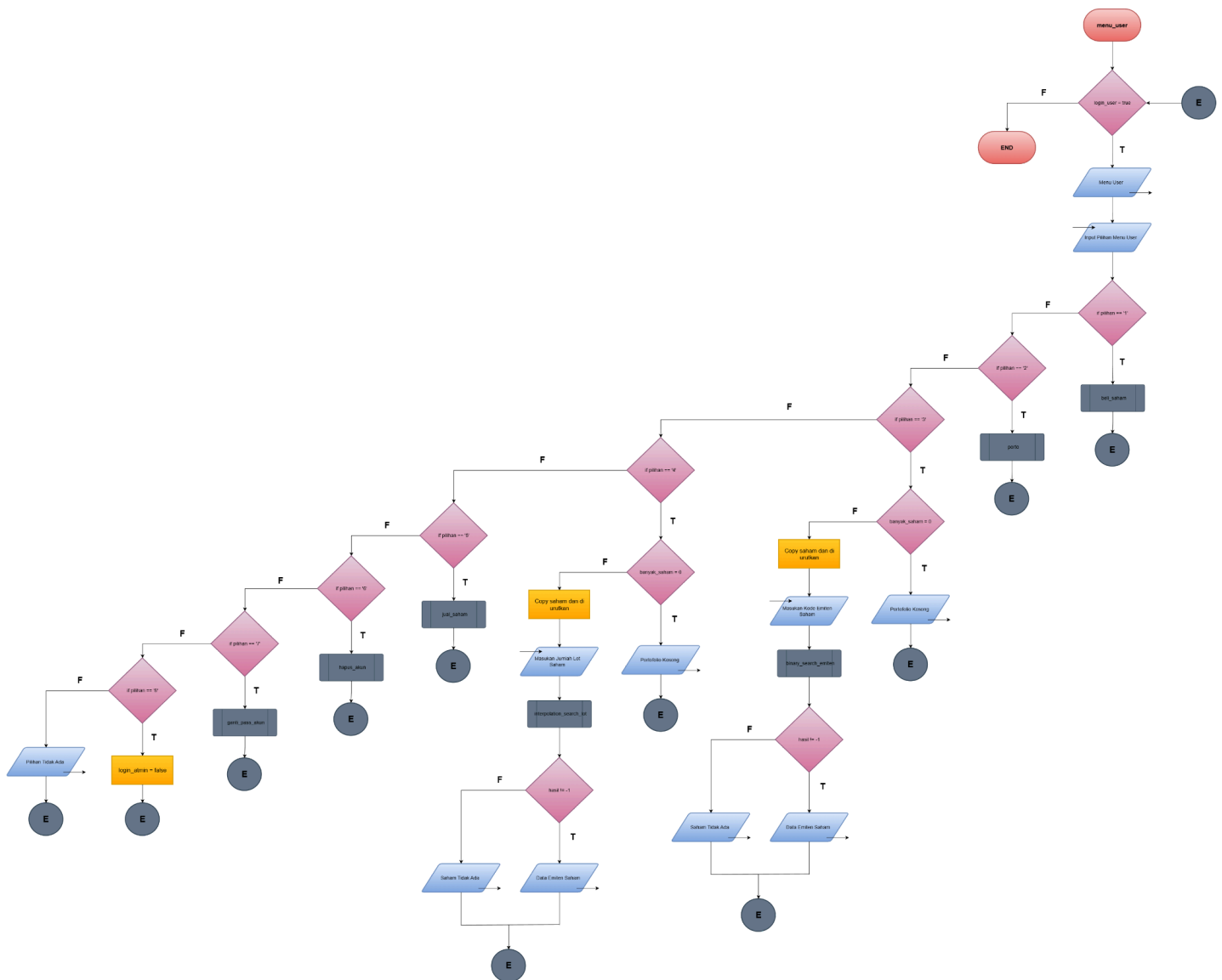


Gambar 1.C. Flowchart Menu Admin

Fungsi ini menggunakan sistem perulangan (**login_atmin = true**) yang akan terus berjalan sampai Admin memilih untuk Logout.

- **Opsi 1 (Lihat Daftar Investor)** → Sistem mengecek variabel **banyak_investor**. Jika masih 0, muncul pesan "**Belum Ada Investor**". Jika ada data, sistem akan menampilkan daftar seluruh investor yang terdaftar
- **Opsi 2 (Cari Akun Investor)** → Mencari akun investor berdasarkan username dari akun investor, Pencarian dilakukan menggunakan **Linear Search**.
- **Opsi 3 (Hapus Akun Investor)** → Sistem akan mencari investor yang sudah melakukan pengajuan hapus akun, Sistem/Program ini dilengkapi oleh **Error Handling** sehingga bisa mencegah **Error** yang tidak diinginkan. (**minta_hapus == true**).
 - Jika ada pengajuan, Admin memasukkan nomor akun.
 - Sistem memvalidasi nomor tersebut. Jika valid, sistem melakukan "**Geser Elemen Array**" ke kiri dan mengurangi jumlah investor (**banyak_investor -= 1**).
- **Opsi 4 (Logout)** → Variabel **login_atmin** diubah jadi **false**. Panah akan kembali ke pengecekan awal, karena statusnya sudah tidak **true**, maka perulangan berhenti dan Admin keluar dari menu ini (kembali ke Menu Utama dengan konektor **D**).

D. Flowchart Menu User



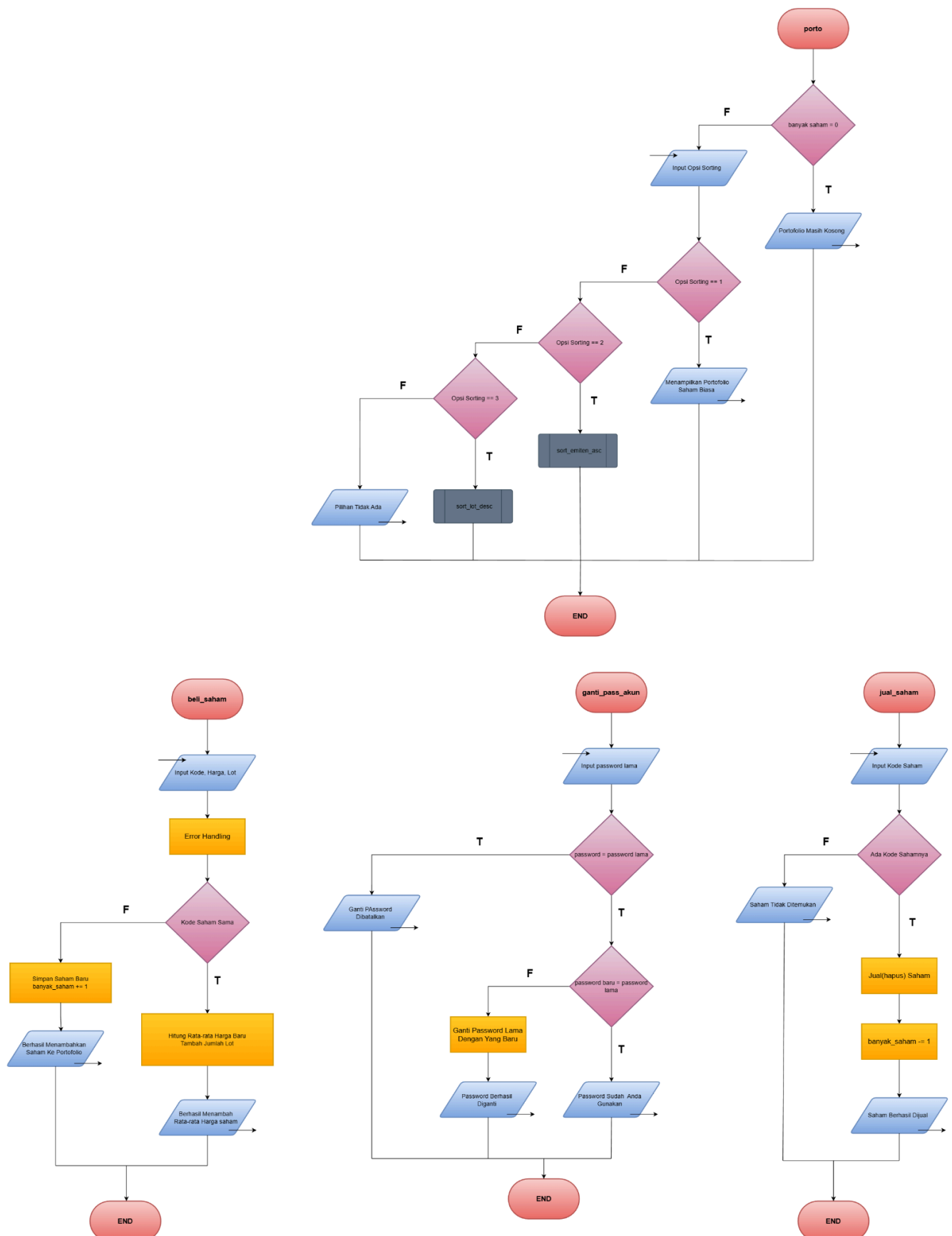
Gambar 1.D. Flowchart Menu User

Sama seperti menu admin, bagian ini dijaga oleh perulangan (**login_user = true**) agar user bisa melakukan banyak transaksi tanpa harus login ulang terus-terusan.

- **Pilihan 1 (Beli Saham):** Memanggil sub-program **beli_saham**. yang otomatis menambahkan rata-rata harga saham jika sudah ada kode saham yang sama.
- **Pilihan 2 (Portofolio):** Memanggil sub-program **porto** untuk melihat daftar saham yang dimiliki beserta total modalnya.
- **Pilihan 3 (Cari Saham Berdasarkan Nama Saham):** Mencari suatu saham dengan nama emiten sahamnya, Pencarian dilakukan dengan menggunakan **Binary Search**.
- **Pilihan 4 (Cari Saham Berdasarkan Jumlah Lot):** Mencari suatu saham berdasarkan jumlah lot suatu saham, Pencarian dilakukan dengan menggunakan **Interpolation Search**.

- **Pilihan 5 (Jual Saham):** Memanggil sub-program **jual_saham** untuk menjual atau menghapus data saham dari daftar.
- **Pilihan 6 (Hapus Akun):** Memanggil sub-program **hapus_akun**. Disini user hanya **mengajukan** (set variable **minta_hapus = true**), bukan langsung hilang akunnya, karena yang eksekusi hapus nanti adalah Admin.
- **Pilihan 7 (Ganti Password Akun):** Memanggil sub-program **ganti_pass_akun**. Untuk mengganti password dari akun user itu sendiri.
- **Pilihan 8 (Logout):** Mengubah status **login_user** jadi **false**, sehingga perulangan berhenti dan user ditendang balik ke Menu Utama dengan konektor **E**.

E. Flowchart Beli Saham, Lihat Saham, Jual Saham dan Ganti Password Akun

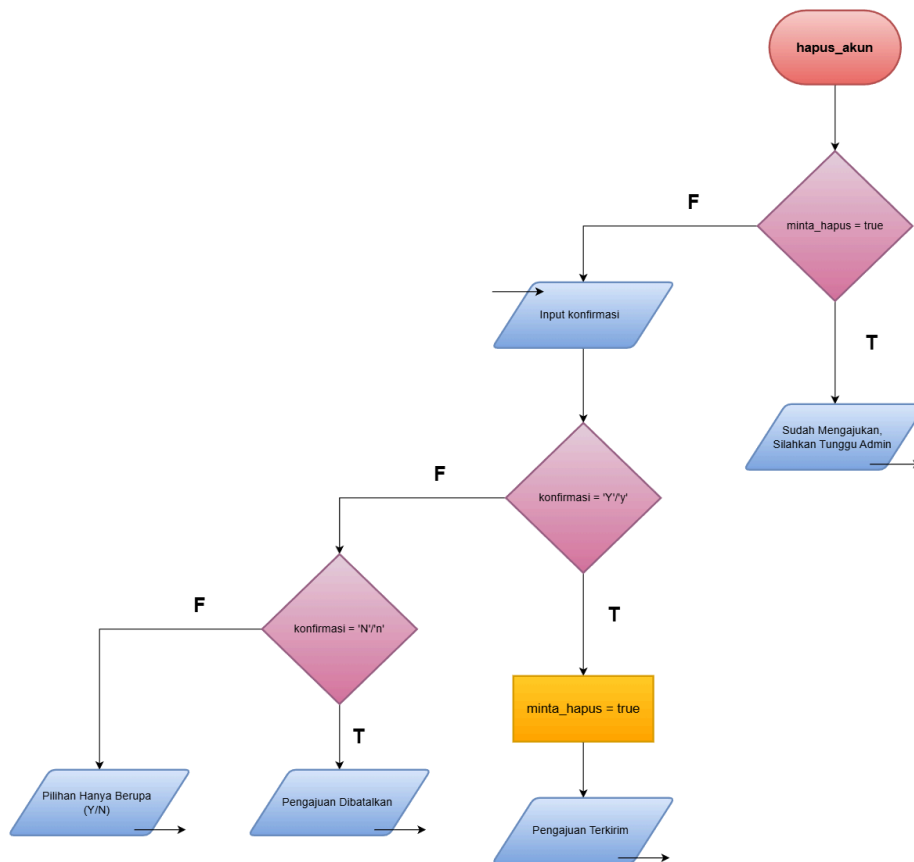


Gambar 1.E. Flowchart Gabungan Beli Saham, Lihat Saham, Jual Saham dan Ganti Password Akun

Flowchart diatas adalah flowchart tiga prosedur yang mengelola portofolio user:

1. **beli_saham**: Berfungsi untuk menambah aset. Sistem ini punya logika → Jika kode saham sudah ada, sistem menghitung harga rata-rata baru dan menambah lot. Jika belum ada, sistem menyimpan data baru dan menambah hitungan **banyak_saham**. Program ini dilengkapi oleh **Error Handling** sehingga bisa mencegah **Error** yang tidak diinginkan.
2. **porto (Portofolio)**: Berfungsi untuk menampilkan aset. Sistem mengecek **banyak_saham**; jika kosong muncul pesan "Masih Kosong", jika ada data saham maka program akan menjalankan sistem "**Pilih**":
 - 1. Mencetak langsung seluruh daftar saham beserta perhitungan **Total Modal**-nya.
 - 2. Mencetak seluruh daftar saham berdasarkan **Huruf Awal** emitennya secara **Ascending** beserta perhitungan **Total Modal**-nya.
 - 3. Mencetak seluruh daftar saham berdasarkan **Banyak Lot Saham** emitennya secara **Descending** beserta perhitungan **Total Modal**-nya.
3. **jual_saham**: Berfungsi untuk menghapus aset. Sistem mencari kode saham yang diinput; jika ketemu, data dihapus dari memori, jumlah **banyak_saham** dikurangi 1, dan memberikan pesan sukses.
4. **ganti_pass_akun**: Berfungsi untuk mengganti password dari akun user itu sendiri, sebelum mengubah password sistem akan meminta konfirmasi berupa password lama user untuk memastikan keaslian user.

F. Flowchart Hapus Akun Investor

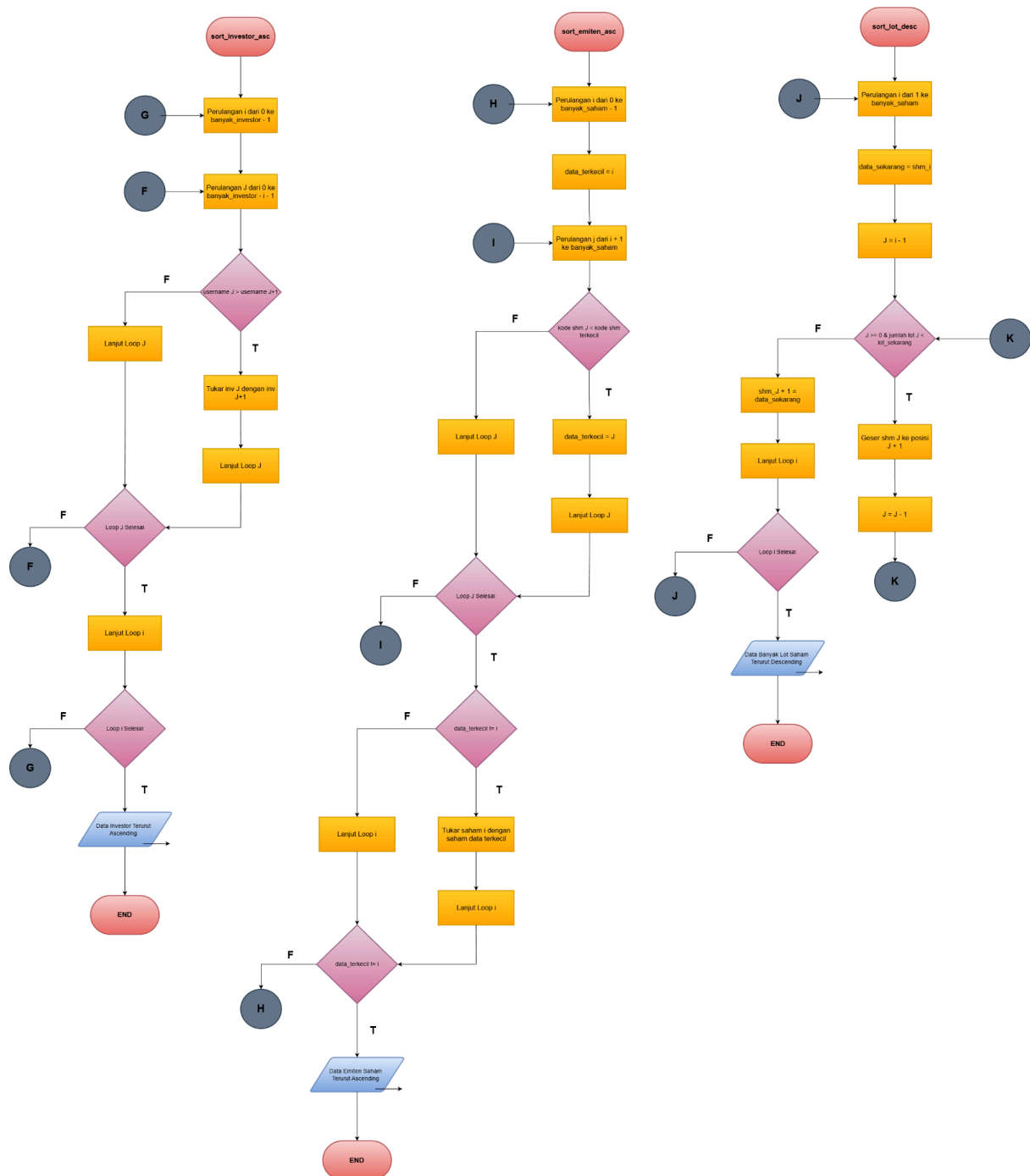


Gambar 1.F Flowchart Hapus Akun Investor

Ini adalah Flowchart prosedur user untuk mengajukan penghapusan akun ke admin jika ingin menghapus akunnya.

- **Filter Awal:** Sistem mengecek dulu, "Kamu udah pernah lapor belum?". Kalau **minta_hapus sudah true**, user cuma disuruh nunggu.
- **Konfirmasi:** Kalau belum, user diminta input konfirmasi (Y/N).
- **Eksekusi:** Jika pilih **Y**, variabel **minta_hapus diubah jadi true** dan tinggal menunggu konfirmasi Admin.

G. Flowchart Sorting Data Investor, Emiten Saham dan Banyak Lot Saham



Gambar 1.G Flowchart Sorting Data Investor, Emiten Saham dan Banyak Lot Saham

1. **sort_investor_asc (Pengurutan Data Investor):** Berfungsi untuk mengurutkan daftar nama investor secara **ascending** (A-Z). Sistem ini menggunakan metode algoritma **Bubble Sort**.

- Logikanya: Sistem melakukan perulangan untuk membandingkan dua **Username** yang letaknya bersebelahan. Jika **Username** sebelah kiri memiliki abjad yang lebih besar daripada **Username** sebelah kanan, maka program akan

menukar posisi keduanya. Pengecekan dan pertukaran ini terus diulang hingga seluruh data nama investor terurut dengan benar.

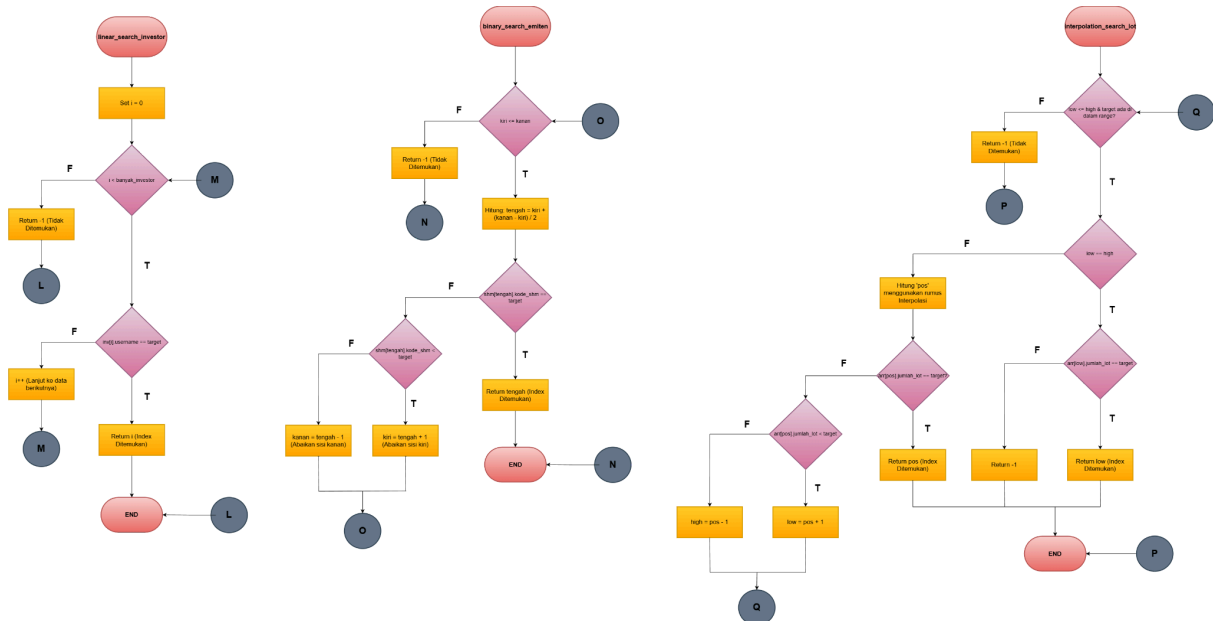
2. **sort_emiten_asc** (Pengurutan Emiten Saham): Berfungsi untuk menampilkan aset portfolio berdasarkan abjad awalan kode emiten secara **ascending** (A-Z). Sistem ini menggunakan metode algoritma Selection Sort.

- Logikanya: Sistem mengasumsikan elemen pada posisi awal sebagai data dengan nilai terkecil (*data_terkecil*). Sistem kemudian memindai sisa data di sebelah kanannya; jika ditemukan kode emiten yang abjadnya lebih kecil, label *data_terkecil* akan dipindahkan ke indeks tersebut. Setelah pemindaian mencapai ujung, barulah sistem menukar elemen awal tadi dengan elemen yang ditunjuk oleh *data_terkecil*.

3. **sort_lot_desc** (Pengurutan Banyak Lot Saham): Berfungsi untuk menampilkan aset portfolio berdasarkan jumlah kepemilikan lot dari yang paling banyak ke paling sedikit (**descending**). Sistem ini menggunakan metode algoritma Insertion Sort.

- Logikanya: Sistem mengambil dan menahan satu data saham ke dalam variabel penampung (*data_sekarang*). Kemudian, sistem membandingkannya dengan data-data di urutan sebelumnya. Jika data sebelumnya memiliki jumlah lot yang lebih kecil, maka data tersebut akan digeser ke kanan untuk mengosongkan ruang. Proses pergeseran berlanjut hingga ditemukan posisi yang tepat, lalu *data_sekarang* akan disisipkan ke ruang kosong tersebut.

H. Flowchart Searching Data Investor, Emiten Saham dan Banyak Lot Saham



Gambar 1.H Flowchart Searching Data Investor, Emiten Saham dan Banyak Lot Saham

1. **linear_search_investor (Pencarian Data Investor):** Berfungsi mencari data akun investor tertentu berdasarkan username. Sistem ini menggunakan algoritma Linear Search.
 - **Logikanya:** Sistem mengecek laci data satu per satu secara berurutan dari awal ($i = 0$). Jika username cocok, program langsung berhenti dan mengembalikan posisi indeksny. Jika seluruh data sudah dicek dan tidak ada yang cocok, program mengembalikan nilai -1 (tidak ditemukan).
2. **binary_search_emen (Pencarian Emiten Saham):** Berfungsi mencari kode emiten saham di dalam portofolio secara cepat. Sistem ini menggunakan algoritma Binary Search dengan syarat data sudah terurut (A-Z).
 - **Logikanya:** Sistem selalu mencari nilai tengah dan membelah data menjadi dua bagian. Jika target abjadnya lebih besar dari nilai tengah, maka setengah data di sebelah kiri langsung dibuang, dan sebaliknya. Proses belah-dua ini terus diulang hingga target ditemukan, atau batas kiri dan kanan saling menyilang (-1).
3. **interpolation_search_lot (Pencarian Banyak Lot):** Berfungsi mencari saham berdasarkan jumlah lot yang dimiliki secara spesifik. Sistem ini menggunakan algoritma Interpolation Search untuk data angka yang sudah terurut.
 - **Logikanya:** Alih-alih mengecek satu-satu atau membelah tengah, sistem menggunakan rumus matematika untuk memprediksi/menebak posisi indeks (pos) berdasarkan besaran nilai target. Jika tebakan meleset, batas pencarian (low atau high) dipersempit hingga target ditemukan, atau berhenti jika target berada di luar jangkauan logika (-1).

2. Deskripsi Singkat Program

Program ini adalah **Aplikasi Pencatatan Portofolio Saham** berbasis C++ yang berfungsi untuk mensimulasikan pencatatan investasi. Logika utamanya mengandalkan struktur data **Array of Struct** ibarat sebuah lemari besar yang menyimpan banyak loker akun User/Investor, dan di dalam setiap loker akun tersebut terdapat laci-laci kecil khusus untuk menyimpan daftar saham.

Secara garis besar, Program ini memiliki 3 peran utama:

1. **Sistem Akun (Registrasi & Autentikasi):** Menangani pembuatan akun baru (maksimal 100 pengguna) dengan fitur pencegah **username** ganda, serta sistem Login yang dibatasi maksimal 3 kali percobaan.
2. **Hak Akses Admin:** Ruang kendali khusus (dengan **username** Jovan) yang mengizinkan admin untuk memantau daftar seluruh investor yang terdaftar di aplikasi.
3. **Manajemen Portofolio Investor (Fitur CRUD):** Ruang pribadi tiap **User** untuk mengelola investasinya. **User** bisa **Beli Saham** (program akan otomatis menghitung ulang harga rata-ratanya jika saham tersebut sudah dimiliki), **Lihat Portofolio** beserta total modalnya, **Jual Semua Saham** untuk menghapus emiten dari daftar, dan **Pengajuan Penghapusan Akun** jika investor ingin menghapus akunnya.

Pada program ini juga ditambah Program sorting berupa:

1. **Bubble Sort** → Untuk menyortir/mengurutkan data para investor berdasarkan huruf awal pada nama mereka secara **Ascending (A-Z)**.
2. **Selection Sort** → Untuk menyortir/mengurutkan data emiten saham yang dimiliki investor berdasarkan huruf awal emitenya secara **Ascending (A-Z)**.
3. **Insertion Sort** → Untuk menyortir/mengurutkan data emiten saham yang dimiliki berdasarkan jumlah lot yang dimiliki dari emiten saham tersebut secara **Descending (Z-A)**.

Program ini juga ditambah program Searching berupa:

1. **Linear Search** → Digunakan untuk mencari data investor berdasarkan username investor.
2. **Binary Search** → Digunakan untuk mencari data emiten saham berdasarkan nama emiten saham.
3. **Interpolation Search** → Digunakan Untuk mencari data emiten saham berdasarkan jumlah lot dari emiten saham.

Program ini juga sudah dilengkapi oleh **Error Handling** sehingga bisa mencegah **Error** yang tidak diinginkan dari beberapa prosedur pada program.

3. Source Code

1. Fondasi Program

```
#include <iostream>
#include <string>
#include <stdexcept>
#include <cctype>
using namespace std;

struct saham {
    string kode_shm;
    int harga_rata_rata;
    int jumlah_lot;
};

struct investor {
    string username;
    string password;
    struct saham shm[50];
    int banyak_saham = 0;
    bool minta_hapus = false;
};
```

2. Fitur Autentikasi dan Kelola Akun

```
bool cek_login(string input_user, string input_pass, string fix_user, string fix_pass){
    return (input_user == fix_user && input_pass == fix_pass);
}

int cek_login(string input_user, string input_pass, investor inv[], int banyak_investor){
    for(int i = 0; i < banyak_investor; i++){
        if(input_user == inv[i].username && input_pass == inv[i].password){
            return i;
        }
    }
    return -1;
}

void buat_akun(investor inv[], int &banyak_investor, string nama_fix){
    string nama, pass;
    cout<<"\n===== "<<endl;
    cout<<"===== PEMBUATAN AKUN CATATAN INVESTOR ===== "<<endl;
    cout<<"===== "<<endl;

    if(banyak_investor >= 100){
        cout<<"TIDAK DAPAT MEMBUAT AKUN KARENA DATA AKUN SUDAH PENUH!!!!"<<endl;
        cout<<"Tekan Enter Untuk Balik Ke Menu Awal...";
        system("pause");
        return;
    }

    cout<<"Masukan Username Anda: ";
    getline(cin, nama);

    bool nama_kembar = false;

    if(nama == nama_fix){
```

```

        nama_kembar = true;
    }
    else{
        for(int i = 0; i < banyak_investor; i++){
            if(nama == inv[i].username){
                nama_kembar = true;
                break;
            }
        }
    }
}

if(nama_kembar){
    cout<<"Username Sudah Terdaftar, Silahkan Buat Username Baru!!!!";
    cout<<"\nTekan Enter Untuk Balik Ke Menu Awal...";
    system("pause");
}
else{
    inv[banyak_investor].username = nama;
    cout<<"Masukan Password Anda: ";
    getline(cin, inv[banyak_investor].password);

    banyak_investor += 1;

    cout<<"\nSelamat Akun Anda Berhasil Dibuat!!!"<<endl;
    system("pause");
}
}

void hapus_akun(investor *user) {
    if(user -> minta_hapus == true){
        cout<<"\n[INFO] Anda Sudah Mengajukan Penghapusan Akun!!!"<<endl;
        cout<<"Silahkan Tunggu Persetujuan Dari Admin :)"<<endl;
    }
    else{
        char konfirmasi;

        cout<<"Apakah Anda Yakin Ingin Mengajukan Penghapusan Akun???(Y/N): ";
        cin>>konfirmasi;
        cin.ignore(100, '\n');

        if(konfirmasi == 'Y' || konfirmasi == 'y'){
            user -> minta_hapus = true;
            cout<<"\n[BERHASIL] Pengajuan Hapus Akun Telah Dikirim Ke Admin!!!"<<endl;
        }
        else if(konfirmasi == 'N' || konfirmasi == 'n'){
            cout<<"\n[BATAL] Pengajuan Dibatalkan."<<endl;
        }
        else{
            cout<<"\nPilihan Hanya Berupa (Y/N) Tidak Bisa Yang Lain!!!!!"<<endl;
        }
    }
}

system("pause");
}

```


3. Fitur Beli Saham, Lihat Saham, Jual Saham, Hapus dan Ganti Password Akun User

```
int total_modal(saham shm[], int n){
    if (n <= 0) {
        return 0;
    }
    int modal_sekarang = shm[n - 1].harga_rata_rata * (shm[n - 1].jumlah_lot * 100);
    return modal_sekarang + total_modal(shm, n - 1);
}

void beli_saham(investor *user) {
    string kode, input_harga, input_lot;
    int harga, lot;
    cout<<"Masukan Kode Emiten (Misal: BBCA): ";
    getline(cin, kode);
    cout<<"Masukan Harga Beli per Lembar: ";
    getline(cin, input_harga);
    cout<<"Masukan Jumlah Lot: ";
    getline(cin, input_lot);

    try{
        if(kode == ""){
            throw invalid_argument("Kode Saham Tidak Boleh Kosong!!!");
        }

        for(int i = 0; i < kode.length(); i++){
            if(!isalpha(kode[i])){
                throw invalid_argument("Kode Saham Hanya Boleh Berupa Huruf!!!");
            }
        }

        try{
            harga = stoi(input_harga);
            lot = stoi(input_lot);

            if(harga <= 0 || lot <= 0){
                throw runtime_error("Harga Saham & Lot Harus Diatas 0!!!");
            }

            int index_saham = -1;
            for(int i = 0; i < user -> banyak_saham; i++){
                if(user -> shm[i].kode_shm == kode){
                    index_saham = i;
                    break;
                }
            }

            if(index_saham != -1){
                int lot_lama = user -> shm[index_saham].jumlah_lot;
                int harga_lama = user -> shm[index_saham].harga_rata_rata;

                int total_modal_lama = lot_lama * harga_lama;
                int total_modal_baru = lot * harga;
                int total_lot = lot_lama + lot;

                user -> shm[index_saham].harga_rata_rata = (total_modal_lama + total_modal_baru) / total_lot;
                user -> shm[index_saham].jumlah_lot = total_lot;

                cout<<"\n[BERHASIL] Anda Berhasil Menambah Rata-rata Harga Saham "<<kode<<"!"<<endl;
            }
        }
    }
```

```

        else{
            int saham_baru = user -> banyak_saham;
            user -> shm[saham_baru].kode_shm = kode;
            user -> shm[saham_baru].harga_rata_rata = harga;
            user -> shm[saham_baru].jumlah_lot = lot;
            user -> banyak_saham += 1;

            cout<<"\n[BERHASIL] Saham "<<kode<<" Berhasil Ditambahkan Ke Portfolio!"<<endl;
        }

    }
    catch(const exception& e){
        cout<<"[ERRRR] "<<e.what()<<endl;
    }
}
catch(const exception& e){
    cout<<"[ERROR] "<<e.what()<<endl;
}

system("pause");
}

void porto(investor *user) {

cout<<"\n===== "<<endl;
    cout<<"===== DAFTAR PORTFOLIO SAHAM
===== "<<endl;

cout<<"===== "<<endl;
    if(user-> banyak_saham == 0){
        cout<<"Portfolio Anda Masih Kosong!"<<endl;
    }
    else{
        int opsi_sort;
        cout<<"\n====Tampilan Portfolio =====<<endl;
        cout<<"1. Default (Sesuai Urutan Beli)"<<endl;
        cout<<"2. Sorting Ascending (Berdasarkan Nama Emiten)"<<endl;
        cout<<"3. Sorting Descending (Berdasarkan Jumlah Lot)"<<endl;
        cout<<"Pilih Opsi Tampilan: ";
        cin>>opsi_sort;
        cout<<"\n";

        if(opsi_sort == 1){
            for(int i = 0; i < user-> banyak_saham; i++){
                string kode = user -> shm[i].kode_shm;
                int harga = user -> shm[i].harga_rata_rata;
                int lot = user -> shm[i].jumlah_lot;
                int modal = harga * (lot * 100);

                cout<< i + 1<<". "<<kode<<" | Lot: "<<lot<<" | Harga Rata-rata: Rp"<<harga<<" | Total Modal:
Rp"<<modal<<endl;
            }

cout<<"----- "<<endl;

            int modal_total = total_modal(user -> shm, user -> banyak_saham);
            cout<<"TOTAL SEMUA MODAL: Rp"<<modal_total<<endl;
        }
        else if(opsi_sort == 2){
            sort_emen_asc(user->shm, user->banyak_saham);

```

```

        for(int i = 0; i < user-> banyak_saham; i++){
            string kode = user -> shm[i].kode_shm;
            int harga = user -> shm[i].harga_rata_rata;
            int lot = user -> shm[i].jumlah_lot;
            int modal = harga * (lot * 100);

            cout<< i + 1<<". "<<kode<<" | Lot: "<<lot<<" | Harga Rata-rata: Rp"<<harga<<" | Total Modal:
Rp"<<modal<<endl;
        }

    cout<<"-----"<<endl;

        int modal_total = total_modal(user -> shm, user -> banyak_saham);
        cout<<"TOTAL SEMUA MODAL: Rp"<<modal_total<<endl;
    }
    else if(opsi_sort == 3){
        sort_lot_desc(user->shm, user->banyak_saham);

        for(int i = 0; i < user-> banyak_saham; i++){
            string kode = user -> shm[i].kode_shm;
            int harga = user -> shm[i].harga_rata_rata;
            int lot = user -> shm[i].jumlah_lot;
            int modal = harga * (lot * 100);

            cout<< i + 1<<". "<<kode<<" | Lot: "<<lot<<" | Harga Rata-rata: Rp"<<harga<<" | Total Modal:
Rp"<<modal<<endl;
        }

    cout<<"-----"<<endl;

        int modal_total = total_modal(user -> shm, user -> banyak_saham);
        cout<<"TOTAL SEMUA MODAL: Rp"<<modal_total<<endl;
    }
    else{
        cout<<"=== Pilihan Tidak Ada!!! ==="<<endl;
    }

}

system("pause");
}

void jual_saham(investor *user) {
    string kode, enter;
    cout<<"Masukan Kode Emiten yang ingin dijual SEMUA: ";
    getline(cin, kode);

    int index_saham = -1;
    for(int i = 0; i < user -> banyak_saham; i++){
        if (user -> shm[i].kode_shm == kode) {
            index_saham = i;
            break;
        }
    }

    if (index_saham != -1){
        for(int i = index_saham; i < user -> banyak_saham - 1; i++){
            user -> shm[i] = user -> shm[i + 1];
        }
        user -> banyak_saham -= 1;
        cout<<"\n[BERHASIL] Saham "<<kode<<" Berhasil Dijual Semua dan Dihapus!!!"<<endl;
    }
}

```

```

else{
    cout<<"\n[GAGAL] Saham "<<kode<<" Tidak Ditemukan Diportfolio Anda."<<endl;
}
cout<<"Tekan Enter Untuk Kembali...";
getline(cin, enter);
}

void ganti_pass_akun(investor *user) {
    string konfirmasi, pass_baru;

    cout<<"Masukan Password Anda Yang Lama: ";
    getline(cin, konfirmasi);

    if(konfirmasi == user -> password){
        cout<<"Masukan Password Anda Yang Baru: ";
        getline(cin, pass_baru);

        if(pass_baru == user -> password){
            cout<<"Password Itu Sudah Anda Gunakan !!!"<<endl;
            system("pause");
        }
        else{
            user -> password = pass_baru;
            cout<<"\n==== Password Anda Berhasil Diganti!!! =====<<endl;
            system("pause");
        }
    }
    else{
        cout<<"\n==== Password Yang Anda Masukan Tidak Sesuai Dengan Password Anda!!! =====<<endl;
        cout<<"===== GANTI PASSWORD DIBATALKAN !!!!! =====<<endl;

        system("pause");
    }
}

```

4. Fitur Tampilan Sub-Menu (Antarmuka)

```

void menu_atmin(investor inv[], int &banyak_investor) {
    bool login_atmin = true;
    char pilihan;
    string enter;

    while(login_atmin) {
        cout<<"\n===== "<<endl;
        cout<<"===== SELAMAT DATANG KEMBALI ATMIN! =====<<endl;
        cout<<"===== "<<endl;
        cout<<"1. Melihat Akun Investor"<<endl;
        cout<<"2. Cari Akun Investor"<<endl;
        cout<<"3. Pengajuan Hapus Akun Investor"<<endl;
        cout<<"4. Logout"<<endl;
        cout<<"\nMasukan Pilihan: ";
        cin>>pilihan; cin.ignore(100, '\n');

        switch(pilihan) {
            case '1':
                cout<<"\n===== "<<endl;
                cout<<"===== DAFTAR INVESTOR TERDAFTAR =====<<endl;
                cout<<"===== "<<endl;

```

```

        if (banyak_investor == 0){
            cout<<"Belum Ada Investor!!!"<<endl;
        }
        else{
            sort_investor_asc(inv, banyak_investor);
            for(int i = 0; i < banyak_investor; i++){
                cout<<i + 1<<". "<<inv[i].username<<endl;
            }
        }

        system("pause");
        break;

    case '2':{
        if (banyak_investor == 0){
            cout<<"Belum Ada Investor Yang Terdaftar!"<<endl;
        }
        else{
            string target;

            cout<<"Masukkan Username Investor Yang Dicari: ";
            getline(cin, target);

            int hasil = linear_search_investor(inv, banyak_investor, target);

            if (hasil != -1) {
                cout<<"\n[DITEMUKAN] Detail Investor:"<<endl;
                cout<<"Username      : "<<inv[hasil].username<<endl;
                cout<<"Total Emiten   : "<<inv[hasil].banyak_saham<<" Saham"<<endl;
                cout<<"Status Hapus    : "<<(inv[hasil].minta_hapus ? "Ya" : "Tidak")<<endl;
            }
            else{
                cout<<"\n[TIDAK DITEMUKAN] Investor Dengan Username:"<<target<<" ,Tidak
Terdaftar!!!"<<endl;
            }
        }

        system("pause");
        break;
    }

    case '3':{
        cout<<"\n===== "<<endl;
        cout<<"===== DAFTAR PENGAJUAN HAPUS AKUN ===== "<<endl;
        cout<<"===== "<<endl;

        int laporan = 0;
        for(int i = 0; i < banyak_investor; i++){
            if(inv[i].minta_hapus == true){
                cout<<i + 1<<". Username: "<<inv[i].username<<" Mengajukan Hapus Akun"<<endl;
                laporan++;
            }
        }

        if(laporan == 0){
            cout<<"Belum Ada Pengajuan Hapus Akun Dari Investor "<<endl;
        }
        else if(laporan > 0){
            int pilihan_hapus;
            string input_hapus;

            cout<<"\nMasukan Nomor Akun Yang Ingin Dihapus (Ketik 0 Untuk Batal): ";
            cin>>input_hapus;

```

```

        cin.ignore(100, '\n');

        try{
            pilihan_hapus = stoi(input_hapus);

            if(pilihan_hapus > 0 && pilihan_hapus <= banyak_investor){
                int idx_investor = pilihan_hapus - 1;

                if(inv[idx_investor].minta_hapus == true){
                    string nama_dihapus = inv[idx_investor].username;

                    for(int i = idx_investor; i < banyak_investor - 1; i++){
                        inv[i] = inv[i + 1];
                    }
                    banyak_investor -= 1;
                    cout<<"\n[BERHASIL] Akun "<<nama_dihapus<<" Berhasil Dihapus!!!"<<endl;
                }
                else{
                    cout<<"\n[GAGAL] Akun Tersebut Tidak Mengajukan Penghapusan Akun!!!"<<endl;
                }
            }
            else if (pilihan_hapus != 0){
                cout << "\n[GAGAL] Nomor tidak valid!" << endl;
            }
        }
        catch(const exception&){
            cout<<"[ERROR] Input Harus Berupa Angka!!!"<<endl;
        }
    }

    system("pause");
    break;
}

case '4':
    login_atmin = false;
    break;

default: cout<<"Pilihan Tidak Ada!!!"<<endl; break;
}
}

}

void menu_user(investor &user) {
    bool login_user = true;
    char pilihan;

    while(login_user) {
        cout<<"\n===== "<<endl;
        cout<<"=== PORTFOLIO INVESTOR: "<<user.username<<" ==="<<endl;
        cout<<"===== "<<endl;
        cout<<"1. Beli Saham"<<endl;
        cout<<"2. Lihat Portfolio Saham"<<endl;
        cout<<"3. Cari Saham Berdasarkan Nama Saham"<<endl;
        cout<<"4. Cari Saham Berdasarkan Jumlah Lot"<<endl;
        cout<<"5. Jual Semua Saham"<<endl;
        cout<<"6. Ajukan Penghapusan Akun"<<endl;
        cout<<"7. Ganti Password Akun"<<endl;
        cout<<"8. Logout"<<endl;
        cout<<"\nMasukan Pilihan: ";
        cin>>pilihan; cin.ignore(100, '\n');

        switch(pilihan) {

```

```

        case '1': beli_saham(&user);
        break;

        case '2': porto(&user);
        break;

        case '3':{
            string target;

            if(user.banyak_saham == 0){
                cout<<"Portfolio Kosong!!!"<<endl;
                system("pause");
                break;
            }
            saham* shm_temp = new saham[user.banyak_saham];

            for(int i = 0; i < user.banyak_saham; i++){
                shm_temp[i] = user.shm[i];
            }

            sort_emen_asc(shm_temp, user.banyak_saham);

            cout<<"Masukkan Kode Emiten Yang Dicari: ";
            getline(cin, target);

            int hasil = binary_search_emen(shm_temp, 0, user.banyak_saham - 1, target);

            if(hasil != -1){
                cout<<"\n[KETEMU] " <<target<<" | Lot: " <<shm_temp[hasil].jumlah_lot<<" | Harga:
Rp" <<shm_temp[hasil].harga_rata_rata<<endl;
            }
            else{
                cout<<"\n[GAGAL] Saham tidak ada!!!"<<endl;
            }

            delete[] shm_temp;
            system("pause");
            break;
        }

        case '4':{
            int target_lot;

            if(user.banyak_saham == 0){
                cout<<"Portfolio Kosong!!!"<<endl;
                system("pause");
                break;
            }
            saham* shm_temp = new saham[user.banyak_saham];

            for(int i = 0; i < user.banyak_saham; i++){
                shm_temp[i] = user.shm[i];
            }

            sort_lot_asc(shm_temp, user.banyak_saham);

            cout<<"Masukkan Target JUMLAH LOT Yang Dicari: ";
            cin>>target_lot;
            cin.ignore(100, '\n');

            int hasil = interpolation_search_lot(shm_temp, 0, user.banyak_saham - 1, target_lot);

            if(hasil != -1){
                cout<<"\n[KETEMU] Ada saham dengan " <<target_lot<<" Lot, Yaitu Saham:

```

```

"<<shm_temp[hasil].kode_shm<<endl;
    }
    else{
        cout<<"\n[GAGAL] Anda Tidak Memiliki Saham Dengan Jumlah Lot:"<<target_lot<<" Lot"<<endl;
    }

    delete[] shm_temp;
    system("pause");
    break;
}

case '5':
    jual_saham(&user);
    break;

case '6':
    hapus_akun(&user);
    break;

case '7':
    ganti_pass_akun(&user);
    break;

case '8':
    login_user = false;
    break;

default:
    cout<<"Pilihan Tidak Ada!!!"<<endl;
    break;
}
}
}

```

5. Program Utama (Main)

```

int main(){
    struct investor inv[100];
    string nama_fix = "Jovan";
    string pass_fix = "031";
    int banyak_investor = 0;
    char pilihan;
    string enter, nama, pass;

    while(true){
        cout<<"\n===== "<<endl;
        cout<<"===== APLIKASI CATATAN INVESTOR ===== "<<endl;
        cout<<"===== "<<endl;
        cout<<"1. Login" << endl;
        cout<<"2. Buat Akun" << endl;
        cout<<"3. Keluar" << endl;
        cout<<"===== " << endl;
        cout<<"\nMasukan Pilihan: ";
        cin>> pilihan;
        cin.ignore(100, '\n');

        switch(pilihan){
            case '1':{
                int percobaan = 0;
                bool berhasil_login = false;

```



```

while(percobaan < 3 && !berhasil_login){
    cout<<"Masukan Username Anda: ";
    getline(cin, nama);
    cout<<"Masukan Password Anda: ";
    getline(cin, pass);

    // Cek Role Admin
    if(cek_login(nama, pass, nama_fix, pass_fix)){
        berhasil_login = true;
        menu_atmin(inv, banyak_investor);
    }
    // Cek Role Investor
    else{
        int index_login = cek_login(nama, pass, inv, banyak_investor);
        if(index_login >= 0){
            berhasil_login = true;
            menu_user(inv[index_login]);
        }
        else{
            bool akun_terdaftar = false;
            for(int i = 0; i < banyak_investor; i++){
                if(inv[i].username == nama){
                    akun_terdaftar = true;
                    break;
                }
            }

            if(nama == nama_fix || akun_terdaftar){
                percobaan += 1;
                cout<<"\nPASSWORD SALAH!!!"<<endl;
                if(percobaan < 3){
                    cout<<"Silahkan Tekan Enter Untuk Menginput Ulang (Sisa Percobaan: "<<3 -
percobaan<<" ) :";

                    getline(cin, enter);
                }
            }
            else{
                cout<<"\nAKUN BELUM TERDAFTAR!!!"<<endl;
                cout<<"Tekan Enter Untuk Balik Ke Menu Awal dan Membuat Akun";
                getline(cin, enter);
                break;
            }
        }
    }
}

if(percobaan == 3){
    cout<<"\nANDA TELAH GAGAL 3X. KEMBALI KE MENU AWAL!!!"<<endl;
    cout<<"Tekan Enter Untuk Kembali Ke Menu Awal: ";
    getline(cin, enter);
}
break;
}
case '2':
    buat_akun(inv, banyak_investor, nama_fix);
    break;
case '3':
    cout<<"\nTerima Kasih Telah Menggunakan Aplikasi Kami :)"<<endl;
    return 0;
default:
    cout<<"Pilihan Tidak Ada!!!"<<endl;
    cout<<"Tekan Enter Untuk Balik Ke Menu Awal: ";
    getline(cin, enter);
    break;
}

```

```

    }
}
return 0;
}

```

6. Program Sorting Data Investor, Huruf Emiten Saham dan Lot Emiten Saham

```

void sort_investor_asc(investor inv[], int banyak_investor){ //<--- Bubble Sort Ascending Untuk Mengurutkan Data Investor Berdasarkan Ururtan Nama (:)
    for(int i = 0; i < banyak_investor - 1; i++) {
        for(int j = 0; j < banyak_investor - i - 1; j++) {
            if(inv[j].username > inv[j+1].username) {
                investor temp = inv[j];
                inv[j] = inv[j+1];
                inv[j+1] = temp;
            }
        }
    }
}

void sort_emiten_asc(saham shm[], int banyak_saham){ //<--- Selection Sort Untuk Mengurutkan Data Emiten Saham Berdasarkan Nama
    for(int i = 0; i < banyak_saham - 1; i++) {
        int data_terkecil = i;

        for(int j = i + 1; j < banyak_saham; j++){
            if(shm[j].kode_shm < shm[data_terkecil].kode_shm){
                data_terkecil = j;
            }
        }

        if(data_terkecil != i){
            saham temp = shm[i];
            shm[i] = shm[data_terkecil];
            shm[data_terkecil] = temp;
        }
    }
}

void sort_lot_desc(saham shm[], int banyak_saham){ //<-- Insertion Sort Untuk Mengurutkan Data Banyak Lot Saham Yang Dimiliki Secara Descending
    for(int i = 1; i < banyak_saham; i++){
        saham data_sekarang = shm[i];
        int j = i - 1;

        while(j >= 0 && shm[j].jumlah_lot < data_sekarang.jumlah_lot){
            shm[j + 1] = shm[j];
            j = j - 1;
        }
        shm[j + 1] = data_sekarang;
    }
}

void sort_lot_asc(saham shm[], int banyak_saham){ //<-- Insertion Sort Tapi Ini Ascending, Karena nanti di search pakai interpolation.
                                                    // (Kalau pakai yang data Desc, rumus Interpolationnya berubah)
    for(int i = 1; i < banyak_saham; i++){
        saham data_sekarang = shm[i];

```

```

        int j = i - 1;

        while(j >= 0 && shm[j].jumlah_lot > data_sekarang.jumlah_lot){
            shm[j + 1] = shm[j];
            j = j - 1;
        }
        shm[j + 1] = data_sekarang;
    }
}

```

7. Program Searching Data Investor, Huruf Emiten Saham dan Lot Emiten Saham

```

int linear_search_investor(investor inv[], int banyak_investor, string target){ //<-- Linear Search Untuk
Mecari data investor tertentu
    for(int i = 0; i < banyak_investor; i++){
        if(inv[i].username == target) {
            return i;
        }
    }
    return -1;
}

int binary_search_emiten(saham shm_temp[], int kiri, int kanan, string target){ //<-- Binary search untuk
mencari emiten saham tertentu
    while (kiri <= kanan){
        int tengah = kiri + (kanan - kiri) / 2;

        if(shm_temp[tengah].kode_shm == target){
            return tengah;
        }
        else if (shm_temp[tengah].kode_shm < target){
            kiri = tengah + 1;
        }
        else{
            kanan = tengah - 1;
        }
    }
    return -1;
}

int interpolation_search_lot(saham arr[], int low, int high, int target){ //<-- Interpolation Search untuk
mencari saham berdasarkan jumlah Lot nya
    while (low <= high && target >= arr[low].jumlah_lot && target <= arr[high].jumlah_lot){
        if (low == high){
            if (arr[low].jumlah_lot == target){
                return low;
            }
            return -1;
        }

        int pos = low + (((double)(high - low) / (arr[high].jumlah_lot - arr[low].jumlah_lot)) * (target -
arr[low].jumlah_lot));

        if(arr[pos].jumlah_lot == target){
            return pos;
        }
        else if(arr[pos].jumlah_lot < target){
            low = pos + 1;
        }
    }
}

```

```

        else{
            high = pos - 1;
        }
    }
    return -1;
}

```

4. Hasil Output

```

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Hah
Masukan Password Anda: Apa
AKUN BELUM TERDAFTAR!!!!
Tekan Enter Untuk Balik Ke Menu Awal dan Membuat Akun

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 

```

Gambar 4.1 Output Menu Utama

Gambar 4.1 adalah contoh Output pada Program ketika, Pengguna memilih **Pilihan ke-1 Menu Utama** dan menginput **Username** yang belum terdaftar pada array akun. Maka pengguna diarahkan untuk membuat akun terlebih dahulu.

```

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Jovan
Masukan Password Anda: 033
PASSWORD SALAH!!!

Silahkan Tekan Enter Untuk Menginput Ulang(Sisa Percobaan: 2) :
Masukan Username Anda: Jovan
Masukan Password Anda: 010
PASSWORD SALAH!!!

Silahkan Tekan Enter Untuk Menginput Ulang(Sisa Percobaan: 1) :
Masukan Username Anda: Jovan
Masukan Password Anda: Hah?
PASSWORD SALAH!!!

Silahkan Tekan Enter Untuk Menginput Ulang(Sisa Percobaan: 0) :

ANDA TELAH GAGAL 3X. KEMBALI KE MENU AWAL!!!!
Tekan Enter Untuk Kembali Ke Menu Awal: 

```

Gambar 4.2 Output Looping Percobaan

Gambar 4.2 adalah contoh Output pada Program ketika, Pengguna memilih **Pilihan ke-1 Menu Utama** dan menginput **Password** yang salah sebanyak 3x, Sehingga program menyatakan bahwa login yang dilakukan oleh pengguna tersebut gagal dan akan balik ke **Menu Utama**

```
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====
Masukan Pilihan: 2
=====
===== PEMBUATAN AKUN CATATAN INVESRTOR =====
Masukan Username Anda: Jovan
Username Sudah Terdaftar, Silahkan Buat Username Baru!!!!
Tekan Enter Untuk Balik Ke Menu Awal...|
```

Gambar 4.3 Output Pembuatan Akun Dengan Username Terdaftar

Gambar 4.3 adalah contoh Output pada Program ketika, Pengguna memilih **Pilihan ke-2 Menu Utama** untuk membuat akun baru dan pengguna menginput **Username** yang sudah ada, Maka program akan memberitahu bahwa username sudah digunakan dan mengembalikan pengguna ke **Menu Utama**.

```
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====
Masukan Pilihan: 2
=====
===== PEMBUATAN AKUN CATATAN INVESRTOR =====
Masukan Username Anda: Udin
Masukan Password Anda: 01

Selamat Akun Anda Berhasil Dibuat!!!
Tekan Enter Untuk Balik Ke Menu Awal: |
```

Gambar 4.4 Output Pembuatan Akun Investor

Gambar 4.4 adalah contoh Output pada Program ketika, Pengguna memilih **Pilihan ke-2 Menu Utama** untuk membuat akun baru dan pengguna menginput **Username** yang belum terdaftar/belum ada pengguna lain yang membuat akun dengan **Username** tersebut, Maka program akan membuat akun baru dari inputan pengguna.

```

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Udin
Masukan Password Anda: 01

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 1
Masukan Kode Emiten (Misal: BBKA): BMRI
Masukan Harga Beli per Lembar: 4900
Masukan Jumlah Lot: 100

[BERHASIL] Saham BMRI Berhasil Ditambahkan Ke Portfolio!
Tekan Enter Untuk Kembali...

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 2

===== DAFTAR PORTFOLIO SAHAM =====
1. BMRI | Lot: 100 | Harga Rata-rata: Rp4900 | Total Modal: Rp4900000
-----
TOTAL SEMUA MODAL: Rp4900000
Tekan Enter Untuk Kembali...

```

Gambar 4.5 Output Beli Saham

Gambar 4.5 adalah contoh Output pada Program ketika pengguna login sebagai “Investor” dengan akun yang sudah dibuat, pada gambar itu Pengguna dengan Akun Investor memilih **Pilihan ke-1 pada Menu Investor** untuk menambah saham yang sudah dibeli dengan format(Emiten, Harga beli per lembar dan Lot), Pada gambar itu pengguna juga memilih **Pilihan ke-2** untuk mengecek daftar saham yang dimilikinya, **di pilihan ke-2 ini User dapat memilih melihat Portofolionya berdasarkan Nama Emiten (Gambar 4.6) dan Banyak Lot Saham dari suatu emiten (Gambar 4.7).**

```

=====
===== DAFTAR PORTFOLIO SAHAM =====
=====

====Tampilan Portfolio ====
1. Default (Sesuai Urutan Beli)
2. Sorting Ascending (Berdasarkan Nama Emiten)
3. Sorting Descending (Berdasarkan Jumlah Lot)
Pilih Opsi Tampilan: 2

1. BBKA | Lot: 1000 | Harga Rata-rata: Rp6780 | Total Modal: Rp678000000
2. BMRI | Lot: 890 | Harga Rata-rata: Rp4700 | Total Modal: Rp418300000
3. BNBR | Lot: 345 | Harga Rata-rata: Rp130 | Total Modal: Rp4485000
4. PTRO | Lot: 447 | Harga Rata-rata: Rp1600 | Total Modal: Rp71520000
-----
TOTAL SEMUA MODAL: Rp1172305000
Press any key to continue . . .

```

Gambar 4.6 Output Portofolio Berdasarkan Nama Emiten Saham

Gambar 4.6 adalah contoh Output pada Program ketika **Investor** memilih untuk melihat portofolio saham yang dimilikinya berdasarkan **Nama Emiten**. Maka Program akan

menampilkan Output seperti pada Gambar 4.6 yang dimana diurutkan secara **Ascending**.

```
=====
===== DAFTAR PORTFOLIO SAHAM =====
=====

====Tampilan Portfolio ====
1. Default (Sesuai Urutan Beli)
2. Sorting Ascending (Berdasarkan Nama Emiten)
3. Sorting Descending (Berdasarkan Jumlah Lot)
Pilih Opsi Tampilan: 3

1. BBKA | Lot: 1000 | Harga Rata-rata: Rp6780 | Total Modal: Rp678000000
2. BMRI | Lot: 890 | Harga Rata-rata: Rp4700 | Total Modal: Rp418300000
3. PTRO | Lot: 447 | Harga Rata-rata: Rp1600 | Total Modal: Rp71520000
4. BNBR | Lot: 345 | Harga Rata-rata: Rp130 | Total Modal: Rp4485000

TOTAL SEMUA MODAL: Rp1172305000
Press any key to continue . . .
```

Gambar 4.7 Output Portofolio Berdasarkan Jumlah Lot Emiten Saham

Gambar 4.7 adalah contoh Output pada Program ketika **Investor** memilih untuk melihat portofolio saham yang dimilikinya berdasarkan **Jumlah Lot Emiten Saham**. Maka Program akan menampilkan Output seperti pada Gambar 4.7 yang dimana diurutkan secara **Descending**.

```
=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portofolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 1
Masukan Kode Emiten (Misal: BBKA): BMRI
Masukan Harga Beli per Lembar: 4800
Masukan Jumlah Lot: 100

[BERHASIL] Anda Berhasil Menambah Rata-rata Saham BMRI!
Tekan Enter Untuk Kembali...

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portofolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 2

===== DAFTAR PORTFOLIO SAHAM =====
1. BMRI | Lot: 200 | Harga Rata-rata: Rp4850 | Total Modal: Rp970000000

TOTAL SEMUA MODAL: Rp970000000
Tekan Enter Untuk Kembali...
```

Gambar 4.8 Output Beli Saham Dengan Emiten Sama

Pada gambar program 4.8 adalah lanjutan dari gambar program sebelumnya yang dimana ketika “**Investor**” membeli saham dengan **Kode Emiten Yang Sama** maka otomatis akan mengupdate harga rata2 beli saham/lembar dan otomatis menambah jumlah lot saham.

```

===== DAFTAR PORTFOLIO SAHAM =====
1. BMRI | Lot: 200 | Harga Rata-rata: Rp4850 | Total Modal: Rp97000000
2. BBNI | Lot: 100 | Harga Rata-rata: Rp3500 | Total Modal: Rp35000000
-----
TOTAL SEMUA MODAL: Rp132000000
Tekan Enter Untuk Kembali...

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 3
Masukan Kode Emiten yang ingin dijual SEMUA: BMNI

[GAGAL] Saham BMNI Tidak Ditemukan Diportfolio Anda.
Tekan Enter Untuk Kembali...

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 3
Masukan Kode Emiten yang ingin dijual SEMUA: BBNI

[BERHASIL] Saham BBNI berhasil Dijual Semua dan Dihapus!!!
Tekan Enter Untuk Kembali...

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 2

===== DAFTAR PORTFOLIO SAHAM =====
1. BMRI | Lot: 200 | Harga Rata-rata: Rp4850 | Total Modal: Rp97000000
-----
TOTAL SEMUA MODAL: Rp97000000
Tekan Enter Untuk Kembali...

```

Gambar 4.9 Output Jual Saham

Gambar 4.9 adalah contoh output lanjutan pada gambar program sebelumnya, Jadi pada gambar program yang ini, Pengguna menambahkan **Emiten baru (BBNI)** seperti yang sudah tertera pada gambar. Kemudian pengguna **Memilih Pilihan Ke-3 Pada Menu Investor** untuk menjual kepemilikan saham **BBNI** nya, Sehingga hanya tersisa saham **BMRI** saja seperti pada gambar di atas.


```
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Udin
Masukan Password Anda: 01

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Logout

Masukan Pilihan: 4
Apakah Anda Yakin Ingin Mengajukan Penghapusan Akun???(Y/N): y

[BERHASIL] Pengajuan Hapus Akun Telah Dikirim Ke Admin!!!
Tekan Enter Untuk Kembali...

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Logout

Masukan Pilihan: 4

[INFO] Anda Sudah Mengajukan Penghapusan Akun!!!
Silahkan Tunggu Persetujuan Dari Admin :)
Tekan Enter Untuk Kembali...
```

Gambar 4.10 Output Pengajuan Hapus Akun

Gambar 4.10 adalah output ketika **Investor** ingin menghapus akunnya, Maka **Investor** harus **Mengajukan** laporan hapus akun kepada admin jika ingin menghapus akun miliknya, Dan ketika Investor sudah **Mengajukan Pesan** maka Investor tidak bisa **Mengajukan Lagi** laporan hapus akun dan harus menunggu tindak lanjut Admin.

```

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 2

===== PEMBUATAN AKUN CATATAN INVESTOR =====
=====
Masukan Username Anda: Udin
Masukan Password Anda: 010

Selamat Akun Anda Berhasil Dibuat!!!
Tekan Enter Untuk Balik Ke Menu Awal:

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Udin
Masukan Password Anda: 010

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfollio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Ganti Password Akun
6. Logout

Masukan Pilihan: 5
Masukan Password Anda Yang Lama: 011

==== Password Yang Anda Masukan Tidak Sesuai Dengan Password Anda!!! ====
===== GANTI PASSWORD DIBATALKAN !!!!! =====
Press any key to continue . . .

```

Gambar 4.11 Output Ganti Password Akun User

Gambar 4.11 program di atas merupakan kondisi ketika pengguna/user memilih untuk mengganti password dari akunnya. **Spesifiknya** pada gambar 4.11 adalah kondisi ketika sistem/program ingin mengkonfirmasi bahwa user yang menggunakan akun adalah user itu sendiri dengan menginputkan password lamanya, jika password yang di input salah, maka akan seperti pada gambar (Gambar 4.11).

```

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Ganti Password Akun
6. Logout

Masukan Pilihan: 5
Masukan Password Anda Yang Lama: 010
Masukan Password Anda Yang Baru: 011

==== Password Anda Berhasil Diganti ====
Press any key to continue . . .

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Ganti Password Akun
6. Logout

Masukan Pilihan: 6

=====
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Udin
Masukan Password Anda: 011

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Ganti Password Akun
6. Logout

Masukan Pilihan:

```

Gambar 4.12 Output Ganti Password Akun User

Gambar 4.12 program di atas merupakan kondisi ketika pengguna/user memilih untuk mengganti password dari akunnya. **Spesifiknya** pada gambar 4.12 adalah kondisi ketika user berhasil mengkonfirmasi kevalidannya/keasliannya sehingga User/Investor dapat mengubah password dari akunnya seperti pada gambar (Gambar 4.12).

```

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Ajukan Penghapusan Akun
5. Ganti Password Akun
6. Logout

Masukan Pilihan: 5
Masukan Password Anda Yang Lama: 010
Masukan Password Anda Yang Baru: 010
Password Itu Sudah Anda Gunakan !!!
Press any key to continue . . .

```

Gambar 4.13 Output Ganti Password Akun User

Gambar 4.13 program di atas merupakan kondisi ketika pengguna/user memilih untuk mengganti password dari akunnya. **Spesifiknya** pada gambar 4.13 adalah kondisi ketika user menginput **password baru sama seperti password lamanya**, maka akan muncul output seperti pada gambar (Gambar 4.13).

```

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 5
Pilihan Tidak Ada!!!

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Jual Semua Saham
4. Logout

Masukan Pilihan: 4

=====
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====
=====
Masukan Pilihan:

```

Gambar 4.14 Output Logout Investor

Gambar 4.14 program di atas merupakan kondisi ketika pengguna memilih **Pilihan** yang tidak tersedia didalam menu dan selanjutnya adalah kondisi ketika pengguna ingin **Logout** dari akunnya.

FITUR BARU USER:

```
=====
===== DAFTAR PORTFOLIO SAHAM =====
=====

====Tampilan Portfolio====
1. Default (Sesuai Urutan Beli)
2. Sorting Ascending (Berdasarkan Nama Emiten)
3. Sorting Descending (Berdasarkan Jumlah Lot)
Pilih Opsi Tampilan: 1

1. BBKA | Lot: 130 | Harga Rata-rata: Rp6700 | Total Modal: Rp87100000
2. BMRI | Lot: 110 | Harga Rata-rata: Rp4500 | Total Modal: Rp49500000
3. BBRI | Lot: 500 | Harga Rata-rata: Rp3250 | Total Modal: Rp162500000
-----
TOTAL SEMUA MODAL: Rp299100000
Press any key to continue . . .

=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Cari Saham Berdasarkan Nama Saham
4. Cari Saham Berdasarkan Jumlah Lot
5. Jual Semua Saham
6. Ajukan Penghapusan Akun
7. Ganti Password Akun
8. Logout

Masukan Pilihan: 3
Masukkan Kode Emiten Yang Dicari: BBRI

[KETEMU] BBRI | Lot: 500 | Harga: Rp3250
Press any key to continue . . .
```

Gambar 4.15 Output Binary Search Berdasarkan Nama Saham

Gambar 4.15 program di atas merupakan output ketika user memilih pilihan untuk mencari(Search) suatu emiten saham berdasarkan **Nama Emiten Sahamnya**.

```
=====
=== PORTFOLIO INVESTOR: Udin ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Cari Saham Berdasarkan Nama Saham
4. Cari Saham Berdasarkan Jumlah Lot
5. Jual Semua Saham
6. Ajukan Penghapusan Akun
7. Ganti Password Akun
8. Logout

Masukan Pilihan: 4
Masukkan Target JUMLAH LOT Yang Dicari: 110

[KETEMU] Ada saham dengan 110 Lot, Yaitu Saham: BMRI
Press any key to continue . . .
```

Gambar 4.16 Output Interpolation Search Berdasarkan Banyak Lot Saham

Masih dengan data saham yang sama, Gambar 4.16 program di atas merupakan output ketika user memilih pilihan untuk mencari(Search) suatu emiten saham berdasarkan **Banyak Dari Lot Sahamnya**.

```

===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Jovan
Masukan Password Anda: 031

===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Pengajuan Hapus Akun Investor
3. Logout

Masukan Pilihan: 1

===== DAFTAR INVESTOR TERDAFTAR =====
=====
1. Username: Andi
2. Username: Lana
3. Username: Siti
4. Username: Zoro
Tekan Enter Untuk Kembali...

```

Gambar 4.17 Output Login Admin

Gambar 4.17 adalah gambar program ketika pengguna **Login** dengan **Akun Admin** dan memilih pilihan **1. Melihat Akun Investor**: Pilihan ini akan menampilkan seluruh akun investor yang terdaftar pada program secara **Ascending**.

```

=====
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Jovan
Masukan Password Anda: 031

=====
===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Pengajuan Hapus Akun Investor
3. Logout

Masukan Pilihan: 1

=====
===== DAFTAR INVESTOR TERDAFTAR =====
=====
1. Username: Udin
2. Username: Budi
Tekan Enter Untuk Kembali...

=====
===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Pengajuan Hapus Akun Investor
3. Logout

Masukan Pilihan: 2

=====
===== DAFTAR PENGAJUAN HAPUS AKUN =====
=====
1. Username: Udin Mengajukan Hapun Akun

Masukan Nomor Akun Yang Ingin Dihapus (Ketik 0 Untuk Batal): 1

[BERHASIL] Akun Udin Berhasil Dihapus!!!
Tekan Enter Untuk Kembali...

=====
===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Pengajuan Hapus Akun Investor
3. Logout

Masukan Pilihan: 1

=====
===== DAFTAR INVESTOR TERDAFTAR =====
=====
1. Username: Budi
Tekan Enter Untuk Kembali...

```

Gambar 4.18 Output Penanganan Hapus Akun Oleh Admin

Gambar **4.18** adalah gambar output ketika ada **Investor** yang mengajukan penghapusan akun, Maka **Admin** akan menangani proses hapus akun investor tersebut dan menghapus akunnya.

FITUR BARU ATMIN:

```
=====
===== APLIKASI CATATAN INVESTOR =====
=====
1. Login
2. Buat Akun
3. Keluar
=====

Masukan Pilihan: 1
Masukan Username Anda: Jovan
Masukan Password Anda: 031

=====
===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Cari Akun Investor
3. Pengajuan Hapus Akun Investor
4. Logout

Masukan Pilihan: 1

=====
===== DAFTAR INVESTOR TERDAFTAR =====
=====
1. Andi
2. Indah
3. Intan
4. Udin
Press any key to continue . . .

=====
===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Cari Akun Investor
3. Pengajuan Hapus Akun Investor
4. Logout

Masukan Pilihan: 2
Masukkan Username Investor Yang Dicari: Andi

[DITEMUKAN] Detail Investor:
Username      : Andi
Total Emiten  : 0 Saham
Status Hapus  : Tidak
Press any key to continue . . .
```

Gambar 4.19 Output Linear Search Berdasarkan Username Investor

Gambar 4.19 program di atas merupakan output ketika admin memilih pilihan untuk mencari(Search) suatu investor berdasarkan Username Investornya.

5. Hasil Output Error

```
===== SELAMAT DATANG KEMBALI ATMIN! =====
=====
1. Melihat Akun Investor
2. Cari Akun Investor
3. Pengajuan Hapus Akun Investor
4. Logout

Masukan Pilihan: 3

===== DAFTAR PENGAJUAN HAPUS AKUN =====
=====
1. Username: VAN Mengajukan Hapus Akun

Masukan Nomor Akun Yang Ingin Dihapus (Ketik 0 Untuk Batal): HAH
[ERROR] Input Harus Berupa Angka!!!
Press any key to continue . . .
```

Gambar 5.1 Output Error Handling Pengajuan Hapus Akun (ADMIN)

Gambar 5.1 adalah gambar output **Error Handling** ketika **Admin** ingin menghapus akun investor yang mengajukan penghapusan akun, tetapi Admin malah **Memasukan Huruf dan Bukan Angka** sehingga menghasilkan output pada Gambar 5.1.

```
=====
=== PORTFOLIO INVESTOR: VAN ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Cari Saham Berdasarkan Nama Saham
4. Cari Saham Berdasarkan Jumlah Lot
5. Jual Semua Saham
6. Ajukan Penghapusan Akun
7. Ganti Password Akun
8. Logout

Masukan Pilihan: 1
Masukan Kode Emiten (Misal: BBKA): A@3%
Masukan Harga Beli per Lembar: 1
Masukan Jumlah Lot: 1
[ERROR] Kode Saham Hanya Boleh Berupa Huruf!!!
Press any key to continue . . .

=====
=== PORTFOLIO INVESTOR: VAN ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Cari Saham Berdasarkan Nama Saham
4. Cari Saham Berdasarkan Jumlah Lot
5. Jual Semua Saham
6. Ajukan Penghapusan Akun
7. Ganti Password Akun
8. Logout

Masukan Pilihan: 1
Masukan Kode Emiten (Misal: BBKA):
Masukan Harga Beli per Lembar: 2
Masukan Jumlah Lot: 2
[ERROR] Kode Saham Tidak Boleh Kosong!!!
Press any key to continue . . .
```

Gambar 5.2 Output Error Handling Beli Saham Emiten (USER)

Gambar 5.2 adalah gambar output **Error Handling** ketika **User/Investor** ingin membeli saham, tetapi menginputkan **Kode Emiten Saham Kosong** atau **Mengandung Selain ALPHABET** sehingga menghasilkan output pada Gambar 5.2.

```

=====
=== PORTFOLIO INVESTOR: VAN ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Cari Saham Berdasarkan Nama Saham
4. Cari Saham Berdasarkan Jumlah Lot
5. Jual Semua Saham
6. Ajukan Penghapusan Akun
7. Ganti Password Akun
8. Logout

Masukan Pilihan: 1
Masukan Kode Emiten (Misal: BBKA): BBKA
Masukan Harga Beli per Lembar: Hah
Masukan Jumlah Lot: 200
[ERORR] invalid stoi argument
Press any key to continue . . .

=====
=== PORTFOLIO INVESTOR: VAN ===
=====
1. Beli Saham
2. Lihat Portfolio Saham
3. Cari Saham Berdasarkan Nama Saham
4. Cari Saham Berdasarkan Jumlah Lot
5. Jual Semua Saham
6. Ajukan Penghapusan Akun
7. Ganti Password Akun
8. Logout

Masukan Pilihan: 1
Masukan Kode Emiten (Misal: BBKA): BBKA
Masukan Harga Beli per Lembar: 200
Masukan Jumlah Lot: HAH
[ERORR] invalid stoi argument
Press any key to continue . . .

```

Gambar 5.3 Output Error Handling Beli Saham Harga Beli & Lot (USER)

Gambar 5.3 adalah gambar output **Error Handling** ketika **User/Investor** ingin membeli saham, tetapi menginputkan **Harga Beli/Jumlah Lot Bukan Berupa Angka** sehingga menghasilkan output pada Gambar 5.3.

6. Langkah-langkah GIT

6.1 GIT Init

```
PS C:\Users\jovan\Praktikum_APL> git init
Reinitialized existing Git repository in C:/Users/jovan/Praktikum_APL/.git/
```

Gambar 6.1 Git Init

Membuat repository lokal di dalam folder yang ingin kita sambungkan ke github

6.2 GIT Add

```
PS C:\Users\jovan\Praktikum_APL> git add .
```

Gambar 6.2 Git Add

Untuk membuat suatu file/Program agar ter-tracking(terdefinisi) oleh git

6.3 GIT Commit

```
PS C:\Users\jovan\Praktikum_APL> git commit -m "Menyelesaikan PT-7 APL"
```

Gambar 6.3 Git Commit

Memberi pesan/tanda terhadap program/file yang baru di buat atau yang baru di ubah

6.4 GIT Remote

```
PS C:\Users\jovan\Praktikum_APL> git remote
```

Gambar 6.4 Git Remote

Menyambungkan/menghubungkan program/file dari git ke github

6.5 GIT Push

```
PS C:\Users\jovan\Praktikum_APL> git push
```

Gambar 6.5 Git Remote

Mengirim program/file yang sudah terinisiasi/ter-tracking ke dalam github